

JC4004 – Computational Intelligence

Sessions 21 & 22: Training deep networks

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

Dr Shahzad Mumtaz (shahzad.mumtaz@abdn.ac.uk)

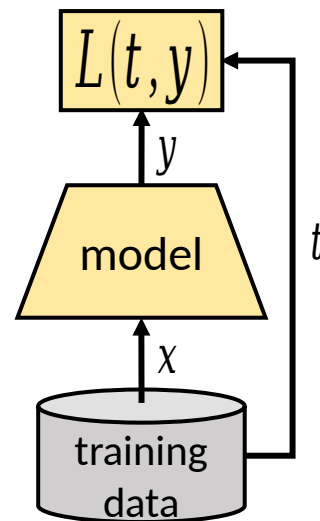
Nov-Dec 2024

How to train deep networks?

- In the earlier lectures, we have introduced the basic algorithms for training deep neural networks, including backpropagation
 - However, there are much more details to consider!
- The main considerations for training deep neural networks:
 - Initialisation of model parameters can have a large impact on training speed and model accuracy
 - The best optimisation algorithm depends on the model and data
 - Properly chosen training parameters and regularisation methods are essential to avoid overfitting

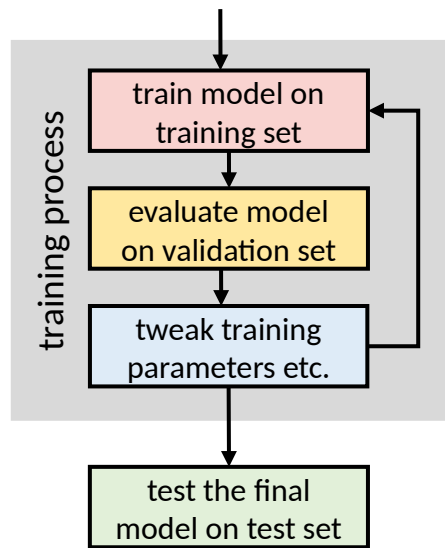
Training data

- Training data contains pairs of input data and the related ground truth label (i.e., target)
 - Small models can be trained using a few hundred or maybe just a few dozens of training samples
 - Deep CNNs for image classification can require millions of training images to achieve good performance!
- Training data is essential for the quality of the model
 - The model can only do what it has been trained for: training data should include all the cases that the final trained model should cover



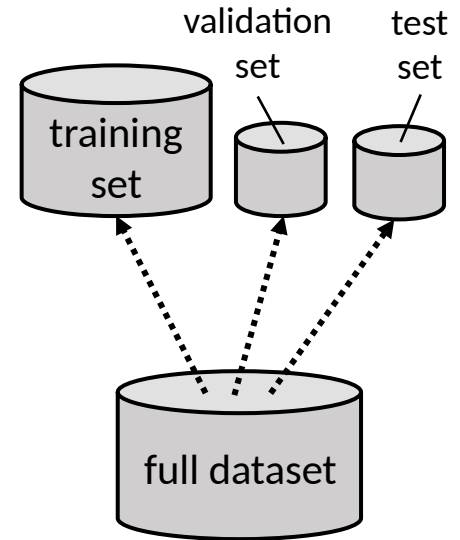
Training, validation, and test sets

- The full dataset is usually divided into training, validation, and test sets
 - *Training set* used for actual training, i.e., updating the model weights during the training process
 - *Validation set* used to validate model performance *during training*
 - *Test set* used to test the final model *after training*
- The subsets need to be representative of the full dataset, and *definitely not overlapping!*



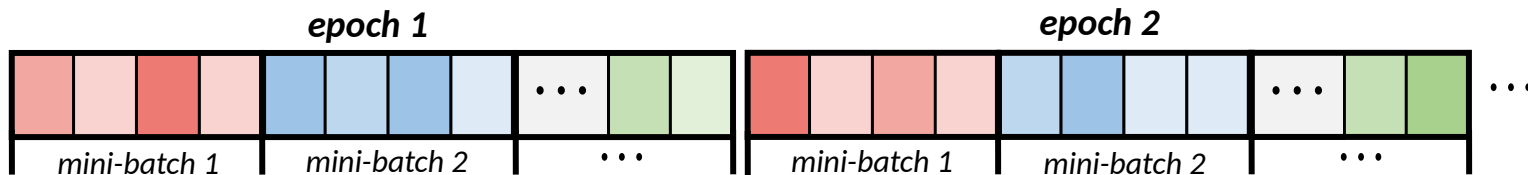
Splitting the dataset

- Some public datasets are divided into training, validation, and test sets by the developers
 - If this is not the case, users must choose the subsets themselves
 - A common practice is to use 60-80% for training, 10-20% for validation, and 10-20% for testing
 - Random split is usually ok, but if the distribution of target classes is skewed, it can be a good idea to preserve class distributions by *stratified sampling*



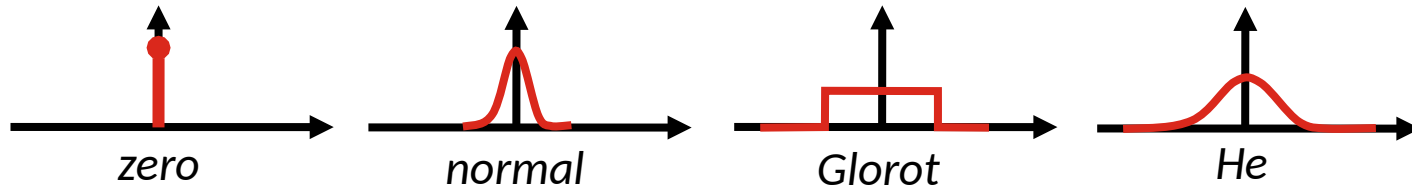
Organisation of training data

- Training data is organised into *epochs*
 - Every training sample appears once in every epoch; often, data is shuffled randomly when a new epoch starts
 - Training takes usually from a few epochs up to hundreds of epochs
- Epochs are further divided into *mini-batches*
 - Gradients for optimisation are computed for one mini-batch at time
 - Optimal mini-batch size depends on data (commonly used size is 16)

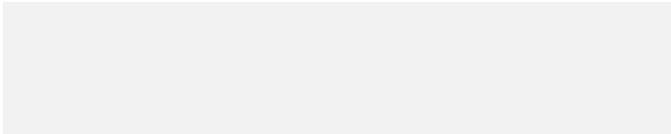
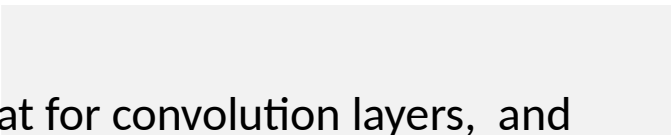


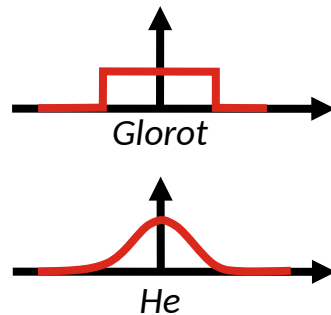
Initialisation

- The initial values of the learnable parameters are set before training
 - Simplest method is *zero initialisation* (the values are set to zeros)
 - Zero initialization often used for biases, but does not work for weights
- Usually, weights initialised using random values
 - *Normal initialisation* uses narrow Gaussian distribution, e.g.,
 - Other commonly used initialisation methods are *Glorot initialisation* (also known as *Xavier initialisation*), and *He initialisation*



Glorot and He initialisation

- Glorot (Xavier) and He initialisations are the most common initialisation methods for convolution and linear layer weights
 - Distributions decided dynamically, based on the layer size
- Given layer input size and output size, the distributions are:
 - Glorot: 
 - He: 
 - Note that for convolution layers, and



Choosing the initialisation method

- Glorot distributions are narrower than He distributions
 - It has been argued that He works better with ReLU activation (and variants), whereas Glorot works better with e.g., Sigmoid and SoftMax
 - It can be a good idea to use different initialisations for different layers
 - It is worth noting that both methods assume (roughly) normalised input, e.g., through batch normalisation layer
- In most cases, the choice between Glorot and He does not make a big difference in the final model performance
 - If you are uncertain, just use the default method of the software library

Optimisation methods

- Deep neural networks are usually trained using different optimisation methods based on gradient descend
 - Gradients of the weights computed through backpropagation and passed to the optimisation algorithm to update the weights
 - *Stochastic gradient descend with momentum (SGDM)* is the baseline method most widely adopted for deep learning
 - Another common method is *adaptive moment estimation (Adam)*
 - Other (rarely used) optimisers include e.g., *RMSProp* and *BFGS*

SGDM optimiser

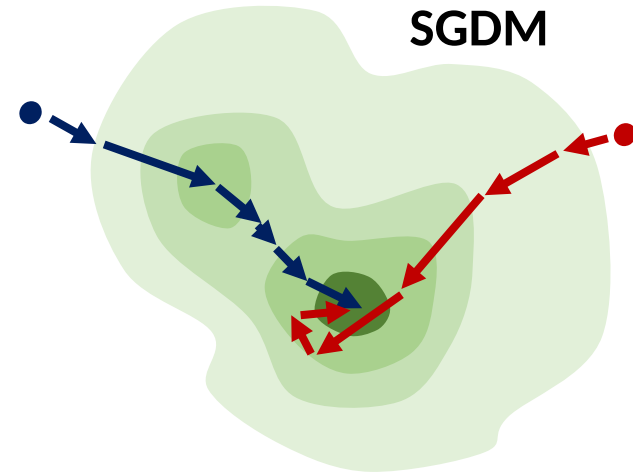
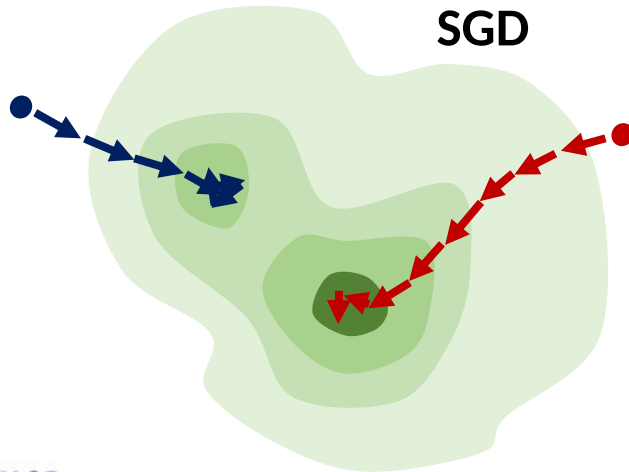
- SGDM is a straightforward extension of SGD, including momentum
 - The updated parameter is computed using the parameter values from two previous iterations and :

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \eta \frac{\partial L}{\partial \theta_i^{(t)}} + \gamma (\theta_i^{(t)} - \theta_i^{(t-1)})$$

- The momentum term accelerates the change when the direction of change is the same for two consecutive iterations
- Parameter γ determines the contribution of momentum

SGD versus SGDM

- SGDM finds the minimum faster than SGD, and due to the momentum term, it does not get trapped as easily in local minima where the gradient is close to zero



Adam optimiser

- Adam optimiser uses moving averages of gradients (i.e., momentum) and squared gradients for more fine-grained adaptation
 - For clarity, we denote gradients as
 - We can express Adam update step as:

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\alpha m_i^{(t)}}{\sqrt{v_i^{(t)} + \epsilon}}$$

Adam optimiser

- Adam optimiser uses moving averages and squared gradients for more fine-grained updates
 - For clarity, we denote gradients as g_t
 - We can express Adam update step as:

Parameters β_1 and β_2 are the decay rates for momentum and squared gradients; typical values are 0.9 and 0.999

Parameter α is the learning rate

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\alpha}{\sqrt{\hat{g}_i^{(t)}}} \hat{g}_i^{(t)}$$

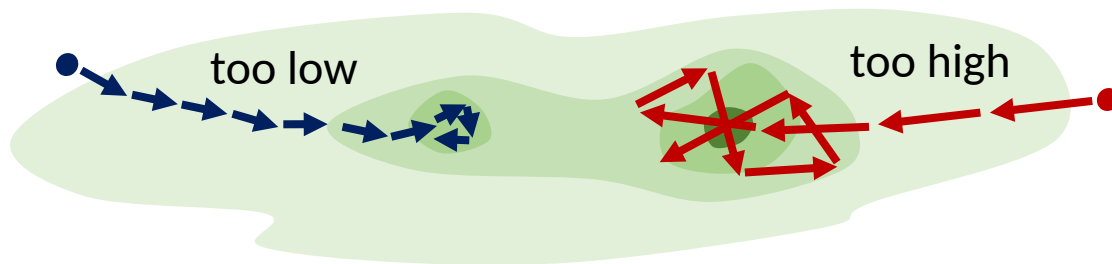
Parameter ϵ is a small constant to avoid division by zero

Choosing the optimiser

- Traditionally, it has been argued that Adam converges faster, but SGDM is more robust to overfitting
 - SGDM is usually the first choice to train CNNs, whereas Adam is the first choice to train transformers (we will talk about transformers later)
 - However, optimiser performance depends a lot on the data and the choice of training parameters: Adam is considered more robust to bad parameter choices
- Other optimisers can be useful for specific types of networks or data
 - For example, BFGS can be useful for small networks with little training data, and RMSprop is often used for training recurrent neural networks

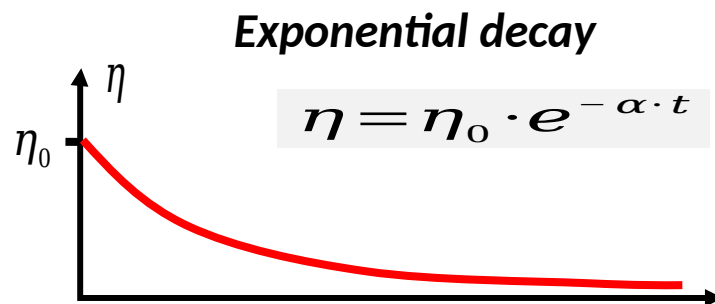
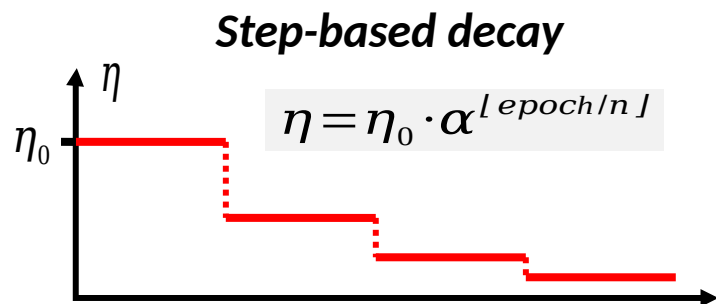
Choosing the learning rate

- Learning rate is the most important parameter for SGDM and Adam
 - Too high learning rate can overshoot the optimal solution, whereas too low learning rate converges slowly and can get stuck in local minima
 - Optimal learning rate depends on the data and the model, and can vary enormously (approximately from 10^{-4} to 10^{-1})
 - Optimal learning rate for Adam tends to be a couple of orders of magnitude smaller than for SGDM, given the same data and model



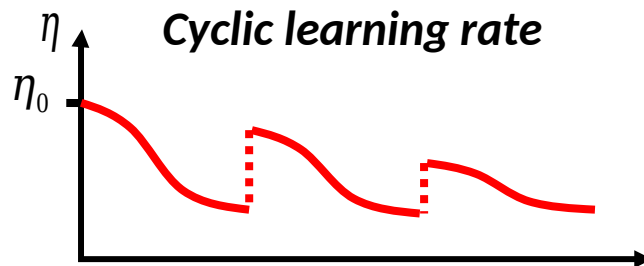
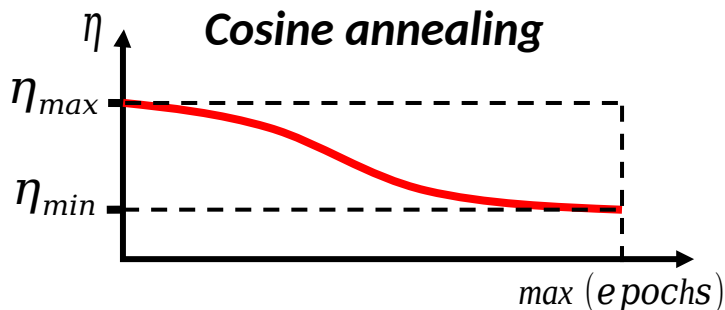
Learning rate scheduler

- *Learning rate schedulers* adjust learning rate during training
 - *Step-based decay* reduces learning rate by a given factor after a fixed number of epochs
 - In *exponential decay*, learning rate decay follows exponential function of time (can be iterations or epochs)



Cosine annealing and cyclic learning rates

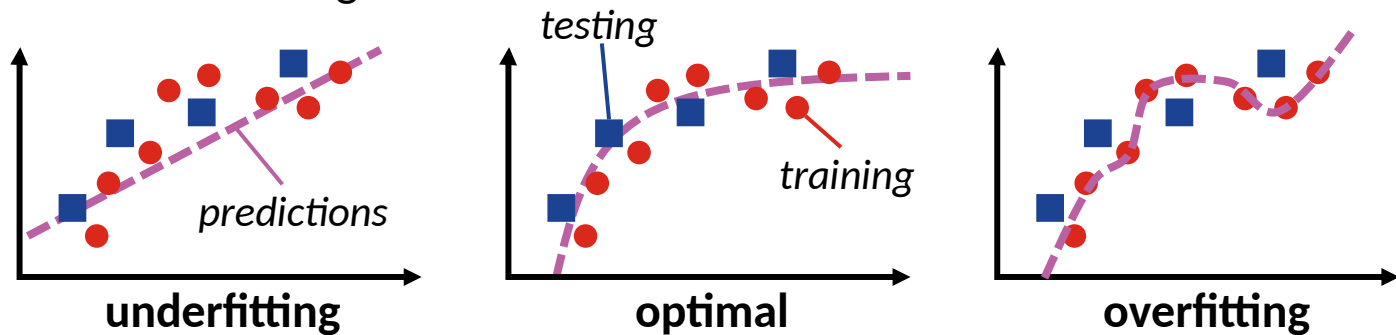
- In *cosine annealing*, learning rate is adjusted by a cosine function
 - Smooth transition from the maximum towards the minimum rate
 - Cosine annealing can be extended to *cyclic learning rates* that can be helpful to avoid learning to be trapped in local plateaus



Break: any questions?

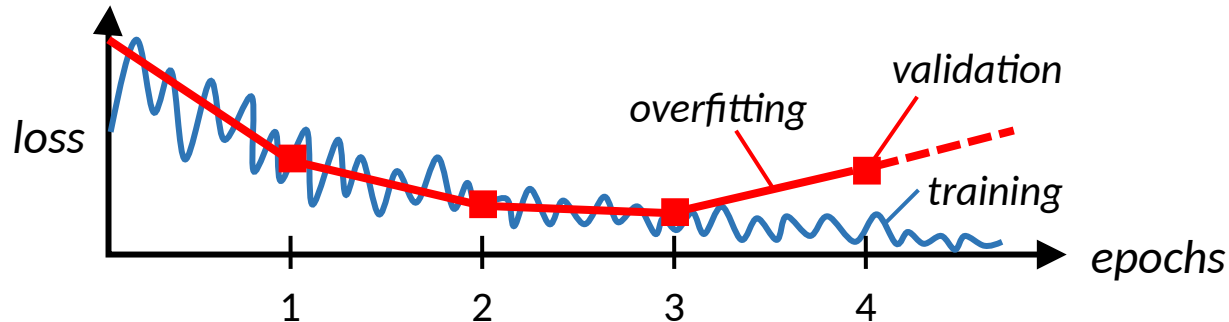
Overfitting

- In *overfitting*, the model learns to predict training data *too accurately*, losing its capability to generalise to unseen data
 - Overfitting occurs especially when there is not enough training data, or the model is too complex for the training data
 - Several different *regularisation techniques* have been developed to avoid overfitting



Early stopping

- The basic method for detecting and avoiding overfitting is to compare training loss and validation loss after each epoch
 - Ideally, validation data loss follows training data loss
 - If validation loss starts to increase, it can indicate overfitting, and training could be stopped, and possibly the model saved after the previous epoch can be used instead of the most recent one



L1 and L2 regularisation

- *L1* and *L2 regularisation* are common techniques in deep learning
 - L1 and L2 regularisations add penalty terms to the loss:

$$L_{L1} = L + \lambda_{L1} \cdot \sum_i |w_i|$$

$$L_{L2} = L + \lambda_{L2} \cdot \sum_i w_i^2$$

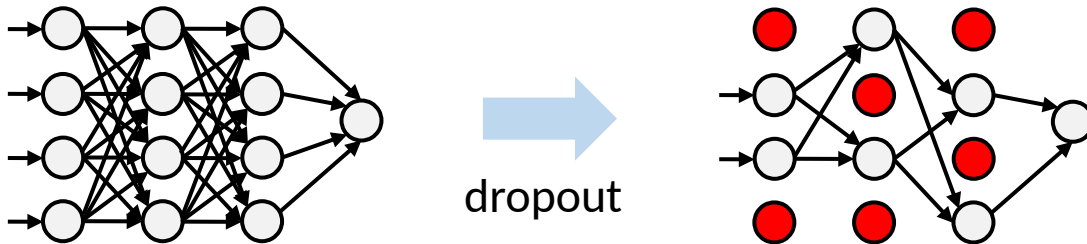
- Hyperparameters λ_{L1} and λ_{L2} control the strength of regularisation
- Note that L1 and L2 regularisations use weights of convolution and linear layers, but not all the learnable parameters (such as biases)!
- Since L and λ are scalars, we can first compute gradients on L and then apply L1 or L2 regularisation

Choosing L1 and L2 regularisation

- L1 regularisation “pushes” weights towards zero: unimportant weights likely to have small values, reducing model complexity
 - Sparsity can be useful if the input data has irrelevant elements
- L2 regularisation decreases large weights more than small weights
 - L2 regularisation is helpful to stabilise and smoothen the model, but does not eliminate irrelevant features completely
 - Usually works well for CNNs, more common than L1 regularisation
- Optimal values for λ_1 and λ_2 depend on data: Commonly used values range from 0 to 1

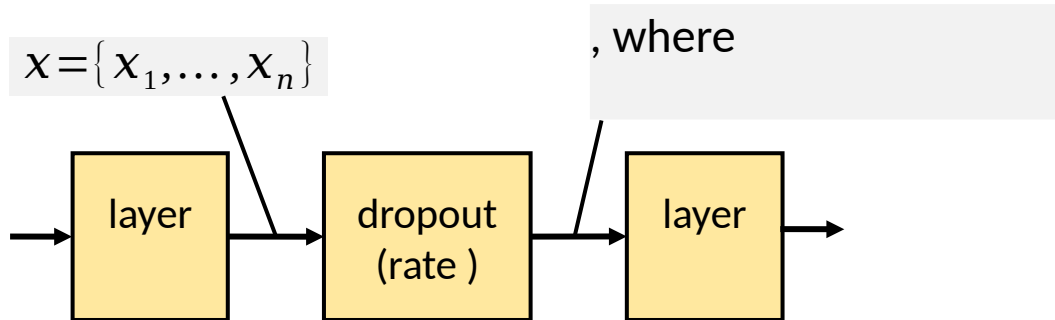
Dropout

- *Dropout* is a regularisation method that sets part of the input values randomly to zeros
 - Dropout helps the model to find alternative paths to reach similar output, making the model more robust
 - Dropout usually implemented by adding a *dropout layer* in the model: dropout layer is only active during training
 - *Dropout rate* (often 0.2...0.5) defines the proportion of zeroed inputs



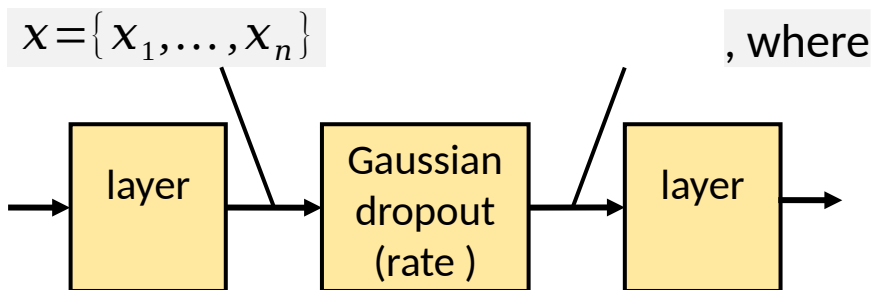
Dropout scaling

- Dropout layer changes the statistical distribution of values propagated through the network!
 - During training, dropout layer should scale the inputs not set to zero by $\frac{1}{p}$ to keep the sum of the input statistically unchanged



Gaussian dropout

- *Gaussian dropout* is an alternative for classic dropout, where Gaussian noise is added to the input
 - Random multiplier follows 1-centered Gaussian distribution
 - Statistical distribution of the values does not change, so there is no need for additional scaling of the inputs



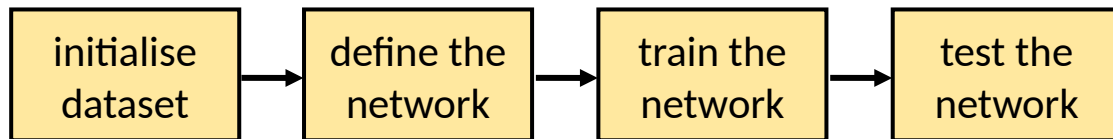
Data augmentation

- One reason for overfitting is that there is not enough training data
 - If the training dataset is small, we can increase the number of training samples by *data augmentation*
 - In data augmentation, additional training samples are generated by different transformations applied to the original samples
 - Note that the ground truth label should remain applicable!



Training deep models in practice

- All the commonly used optimisers, training options, and regularisation techniques are implemented in the prominent deep learning libraries
 - No need to reinvent the wheel!
 - In the following, we go through an example of training an image classification model using PyTorch
 - The example is from PyTorch tutorial (<https://pytorch.org/tutorials/>) with some modifications



Load datasets

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
4
5 transform = transforms.Compose(
6     [transforms.ToTensor(),
7      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
8
9 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
10                                         download=True, transform=transform)
11
12 trainloader = torch.utils.data.DataLoader(trainset, batch_size= 4,
13                                           shuffle=True, num_workers=2)
14
15 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
16                                         download=True, transform=transform)
17
18 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
19                                           shuffle=False, num_workers=2)
```

Load the necessary packages

Define 3-channel
image conversion to
tensors normalised
to range [-1,1]

Load datasets

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
4
5 transform = transforms.Compose(
6     [transforms.ToTensor(),
7      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
8
9 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
10                                         download=True, transform=transform)
11
12 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
13                                           shuffle=True, num_workers=2)
14
15 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
16                                         download=True, transform=transform)
17
18 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
19                                         shuffle=False, num_workers=2)
```

Download CIFAR10 training set from torchvision repository and assign the transform we defined

Define mini-batch size

Use random shuffling of epochs

Create loader for the training dataset

Load datasets

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
4
5 transform = transforms.Compose(
6     [transforms.ToTensor(),
7      transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))]
8 )
9
10 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
11                                         download=True, transform=transform)
12
13 trainloader = torch.utils.data.DataLoader(trainset, batch_size= 4,
14                                           shuffle=True, num_workers=2)
15
16 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
17                                         download=True, transform=transform)
18
19 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
20                                          shuffle=False, num_workers=2)
```

Download CIFAR10 test set and create a loader for it in a same way as for the training set

Create a classification network

```
20 import torch.nn as nn
21 import torch.nn.functional as F
22 class Net(nn.Module):
23     def __init__(self):
24         super().__init__()
25         self.conv1 = nn.Conv2d(3, 6, 5)
26         self.pool = nn.MaxPool2d(2, 2)
27         self.conv2 = nn.Conv2d(6, 16, 5)
28         self.fc1 = nn.Linear(16 * 5 * 5, 120)
29         self.fc2 = nn.Linear(120, 84)
30         self.fc3 = nn.Linear(84, 10)
31     def forward(self, x):
32         x = self.pool(F.relu(self.conv1(x)))
33         x = self.pool(F.relu(self.conv2(x)))
34         x = torch.flatten(x, 1)
35         x = F.relu(self.fc1(x))
36         x = F.relu(self.fc2(x))
37         x = self.fc3(x)
38         return x
39 net = Net()
```

Load more necessary packages

Define class for the CNN, inherited from `nn.Module`

Define layers for the CNN

Define forward pass function for the CNN

Instantiate the CNN

Train the network

```
40 import torch.optim as optim
41
42 optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
43 for epoch in range(2):
44     run_loss = 0.0
45     for i, data in enumerate(trainloader, 0):
46         inputs, labels = data
47         optimizer.zero_grad()
48         outputs = net(inputs)
49         loss = nn.CrossEntropyLoss(outputs, labels)
50         loss.backward()
51         optimizer.step()
52         run_loss += loss.item()
53     if i % 2000 == 1999:
54         print(f'[{epoch+1}, {i+1:5d}] loss: {run_loss/2000:.3f}')
55     run_loss = 0.0
```

Load package for optimisers

Choose SGDM optimiser
and set the learning
parameters

Loop for 2 epochs

Train the network

```
40 import torch.optim as optim
41
42 optimizer = optim.SGD(net.parameters(), lr=0.001,
43 for epoch in range(2):
44     run_loss = 0.0
45     for i, data in enumerate(trainloader, 0):
46         inputs, labels = data
47         optimizer.zero_grad()
48         outputs = net(inputs)
49         loss = nn.CrossEntropyLoss(outputs, labels)
50         loss.backward()
51         optimizer.step()
52         run_loss += loss.item()
53     if i % 2000 == 1999:
54         print(f'[{epoch+1}, {i+1:5d}] loss: {run_loss/2000:.3f}')
55         run_loss = 0.0
```

Loop through all the mini-batches in the training set using trainloader; note that data contains both input and labels!

Reset gradients

Forward pass

Train the network

```
40 import torch.optim as optim
41
42 optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
43 for epoch in range(2):
44     run_loss = 0.0
45     for i, data in enumerate(tr
46         inputs, labels = data
47         optimizer.zero_grad()
48         outputs = net(inputs)
49         loss = nn.CrossEntropyLoss(outputs, labels)
50         loss.backward()
51         optimizer.step()
52     run_loss += loss.item()
53     if i % 2000 == 1999:
54         print(f'[{epoch+1}, {i+1:5d}] loss: {run_loss}
55         run_loss = 0.0
```

Compute loss (note that
CrossEntropyLoss
includes SoftMax!)

Compute gradients
via backpropagation

Update weights
using the optimiser

Train the network

```
40 import torch.optim as optim
41
42 optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
43 for epoch in range(2):
44     run_loss = 0.0
45     for i, data in enumerate(trainloader, 0):
46         inputs, labels = data
47         optimizer.zero_grad()
48         outputs = net(inputs)
49         loss = nn.CrossEntropyLoss(outputs, labels)
50         loss.backward()
51         optimizer.step()
52         run_loss += loss.item()
53     if i % 2000 == 1999:
54         print(f'[epoch+1], {i+1:5d} loss: {run_loss/2000:.3f}')
55         run_loss = 0.0
```

Compute cumulative training loss for every 2000 iterations and print it to the console (note that we do not use a validation set!)

Test the network

```
56 correct = 0 }
57 total = 0
58
59 with torch.no_grad():
60     for data in testloader:
61         images, labels = data
62         outputs = net(images)
63         _, predicted = torch.max(outputs.data, 1)
64         total += labels.size(0)
65         correct += (predicted == labels).sum().item()
66
67 print(f'Accuracy of the network: {100 * correct // total} %')
```

Initialise counters

Gradients not needed for testing

Loop through testing data using test loader

Test the network

```
56 correct = 0
57 total = 0
58
59 with torch.no_grad():
60     for data in testloader:
61         images, labels = data
62         outputs = net(images)
63         _, predicted = torch.max(outputs.data, 1)
64         total += labels.size(0)
65         correct += (predicted == labels).sum().item()
66
67 print(f'Accuracy: {100 * correct // total} %')
```

Get the network outputs for a mini-batch

The predicted label is the index of the output vector with the highest value

Update the counters

Print out the result

Accuracy : 54 %

Summary

- There are a lot of training parameters and details to consider when training deep neural networks
 - Proper selection of training, validation, and test sets: all the subsets should be representative of the problem, but not overlapping
 - Selection of the initialisation method: Glorot and He most common
 - Selection of the optimiser: SGDM and Adam are the most common
 - Selection of the learning rate and rate scheduler
 - Regularisation to avoid overfitting: early stopping, L1 and L2 regularisations, dropout, data augmentation

Questions and discussion